

Planning & Estimation

Estimation is one of the hardest problems in software. We're asked to predict the future for work we've never done before, in systems we don't fully understand.

Why Estimation is Hard

Software estimation fails for predictable reasons:

- **Uncertainty** — We're building something new; if we'd built it before, we'd reuse it
- **Unknown unknowns** — We don't know what we don't know until we start
- **Complexity** — Small changes can have large ripple effects
- **Optimism** — We consistently underestimate how long things take

The problem isn't that developers are bad at estimating. It's that accurate estimation of novel work is fundamentally difficult.

Iterative Planning

Since we can't predict the future accurately, we plan in short cycles:

1. **Plan a small chunk** — One or two weeks of work
2. **Do the work** — Learn what's actually involved
3. **Review and adjust** — Update plans based on what you learned
4. **Repeat**

This is the heart of Agile planning. Instead of a detailed 6-month plan that will be wrong, you have a rough direction and detailed short-term plans that adapt.

Relative Sizing

Many teams use story points instead of hours:

- Compare tasks to each other, not to absolute time
- "This task is about twice as complex as that one"
- Track velocity (points completed per iteration) to forecast capacity

This works because we're better at relative comparison than absolute estimation.

Commitment vs Forecast

There's a difference between:

- **Forecast** — "Based on our velocity, we'll probably finish around March"
- **Commitment** — "We guarantee delivery by March 1st"

Forecasts are honest; commitments under uncertainty are often wishful thinking.

Cargo Cult Planning

Watch out for teams that do Agile rituals without understanding them:

- **Sprints without iteration** — Fixed 2-week cycles, but no adjustment based on learning
- **Planning poker without discussion** — The point is the conversation, not the numbers
- **Velocity as a target** — Velocity measures capacity, not performance. Gaming it helps no one
- **Estimates as commitments** — Treating rough guesses as promises

The ceremonies exist to support the values. If you're doing the ceremonies without the values, you're missing the point.

When Plans Change

Plans will change. Requirements shift, priorities change, you learn things that invalidate assumptions.

This is normal, not failure. The Agile Manifesto says: “Responding to change over following a plan.”

The question isn’t whether plans will change, but how gracefully you adapt when they do.